

1. Objetivo del programa:

- simular selección natural de comportamientos egoístas en una población
- la idea es mostrar que al no cooperar y agarrar recursos para sí mismos, los egoístas sobreviven y se reproducen de forma más rápida, mientras los altruistas que ayudan a los otros se mueren mas rapido
- La idea es que al final de la simulación queden más personas egoístas y menos altruistas.

2. Inputs:

- cantidad de personas altruistas
- cantidad de personas egoístas
- cantidad de rondas de la simulación
- que gráficos se quieren ver

3. Outputs: gráficos

- **Gráfico 1:**
 - Gráfico de líneas: Evolución de la población por turno Tiene dos líneas: una azul para altruistas y una roja para egoístas. Cada punto en la línea muestra cuántos individuos de ese tipo había en ese turno. Sirve para ver si una condición crece, decrece o desaparece a lo largo de la simulación.
- **Gráfico 2:**
 - Gráfico de torta: Distribución final Tiene dos porciones: azul para altruistas y roja para egoístas. Muestra en porcentaje cómo quedó repartida la población al terminar la simulación. Es el resultado principal: qué condición dominó al final.
- **Gráfico 3:**
 - Gráfico de torta: Distribución inicial Igual que el gráfico 2 pero con los datos del primer turno. Sirve para compararlo con el gráfico 2 y ver cuánto cambió la proporción entre el inicio y el final de la simulación.
- **Gráfico 4:**
 - Gráfico de barras: Interacciones por tipo, Tiene cuatro barras: AA, EE, EA y parientes. La altura de cada barra muestra cuántas veces ocurrió ese tipo de encuentro durante toda la simulación. Sirve para ver qué combinación fue más frecuente y si eso tuvo relación con el resultado final.
- **Gráfico 5:**
 - Gráfico de barras: Recursos promedio al final Tiene dos barras, una azul para altruistas y una roja para egoístas. La altura de cada barra muestra el promedio de recursos que tiene cada individuo de esa condición al terminar la simulación. Sirve para ver cuál de las dos condiciones terminó acumulando más recursos.
- **Gráfico 6:**
 - Gráfico de barras: Reproducciones por condición. Tiene dos barras, una azul para altruistas y una roja para egoístas. La altura de cada barra muestra el

total de reproducciones que tuvo cada condición durante toda la simulación. Sirve para ver qué condición se reproduce más a lo largo del juego.

- **Gráfico 7:**

- Gráfico de barras: Muertes por condición. Tiene dos barras, una azul para altruistas y una roja para egoístas. La altura de cada barra muestra el total de muertes que tuvo cada condición durante toda la simulación. Sirve para ver qué condición tuvo más bajas y relacionarlo con el resultado final.

4. Procesos principales + módulos:

1. Pedir cantidades al usuario
 - a. ⇒ main
2. Validar Información
 - a. ⇒ validación
3. Crear las personas y población
 - a. ⇒ simulación
4. Hacer simulación
 - a. ⇒ simulación
5. Mostrar menu de opciones de graficos posibles
 - a. ⇒ main
6. Pedir opción deseada (o salida)
 - a. ⇒ main
7. Validar opción
 - a. ⇒ validación
8. Crear gráfico
 - a. ⇒ gráficos
9. Mostrar gráficos
 - a. main

5. Estructura de datos:

- Clase Persona (objetos)
 - Atributos
 - condición de egoísta o altruísta
 - True / False
 - "ID"
 - int
 - recursos (como sus "vidas")
 - int
 - empiezan con (40),
 - Métodos:
 - Reproducción
 - crea un objeto persona con mismo ID y misma condición (ej: un egoísta crea a otra persona egoísta)
- Clase Poblacion
 - Atributos:
 - personas
 - lista de objetos de todas las personas
 - datos
 - dataframe que registra información de turnos

- ["turno" , "cant_egoistas" , "cant_altruistas" , "total_personas" , "cant_interacciones_EE" , "cant_interacciones_EA" , "cant_interacciones_AA" , "cant_interacciones_parientes" , "cant_reproducciones_E" , "cant_reproducciones_A" , "cant_muertes_E" , "cant_muertes_A"]
 - (así puse el spyder el orden y nombre de las columnas)

◦ **Agregar columnas: recursos_totales_A y recursos_totales_E**

turno	cant_egoistas	cant_altruistas	cant_interacciones_EE	cant_interacciones_EA	cant_interacciones_AA	cant_reproducciones_A	cant_reproducciones_E	total_personas
1								

◦ **Métodos:**

- agregar_personas
 - agrega las personas creadas
- muerte
 - saca persona de la lista de la población → **aca va un try/except para atajar error si es que el elemento a eliminar no está en la lista**
- ronda_interaccion → **acá hay que tener en cuenta que si la lista de población es 0 o 1 termina la simulación(condición de corte)**
 - empareja aleatoriamente dos personas de la población, y depende la condición de altruista/egoísta distribuye recursos.
 - importante:
 - si las personas son pares, hacer de a parejas
 - si las personas son impares, dejar 1 aleatoriamente afuera y después hacer parejas de a dos simultáneamente.
 - intercambio de recursos
 - si interactúa un Altruista con otro altruista: sumar (ej: 1) a cada uno
 - si interactúa egoísta con egoísta no hay cambio
 - pero, si son parientes (el "ID" es el mismo), suman lo mismo que sumaria altruista con altruista. (ej: 1 para cada uno) **validar que el ID coincida.**
 - Si interactúa altruista y egoísta, el altruista le da unos recursos al egoísta. Esto significa que se le suman (ej 5) recursos egoístas , y al altruista se le restan.
- filtrar población
 - se fija en los recursos de cada persona a ver si alguien muere o alguien se reproduce
- agregar_datos_turno
 - agrega los datos del turno necesarios al dataframe

6. Módulos/funciones:

- validar_input (cant_altruistas, cant_egoistas, cant_turnos)
 - tiene que poder convertir inputs en número
 - **error:** si el dato ingresa NO es entero → raise "TypeError"
 - dentro de rango permitido
 - **error:** si el dato ingresado por el usuario supera el rango permitido → raise "ValueError"
 - **error:** si el numero ingresado es cero o negativo → raise "ValueError"
 - **error:** si ambas cantidades son 0 al mismo tiempo (población vacía) → raise "ValueError"
- crear_personas (cant_altruistas_validada , cant_egoista_validada)
 - crea objetos de persona con condición
 - agrega (llamando al método de Población) a la lista de población
- simulación (cant_rondas)
- validar_eleccion_graficos
 - valida que este en el rango de numeros posibles
 - **error:** si el número ingresado es distinto a los que corresponden a cada gráfico → raise "ValueError"
 - **error:** si el tipo de dato ingresado NO es correspondiente → raise "TypeError"
- una función por gráfico para crearlos
- todos los métodos de los objetos